AGGREGATE REVIEW PACKET

Warren Design Review

A consolidated review page for the Warren concept: a central hub for supervising distributed Claude Code sessions inside remote tmux environments, with dedicated session, notification, plugin, and permission management surfaces.

Contents

1. Overview

Warren is a central hub for supervising and interacting with distributed coding-agent sessions that run inside tmux across multiple remote servers.

The immediate problem is not deployment or orchestration in the abstract. The real problem is that a single working environment now spans many remote machines, many long-lived agent sessions, many plugin configurations, and many permission states. Today that workspace is fragmented across SSH logins and tmux windows. Warren pulls it back into one coherent control surface.

At its core, Warren does four things:

1. tracks where agent sessions live,
2. captures what those sessions are doing,
3. classifies when they need user attention,

4. lets the user act on them from a central hub.

The design is intentionally layered. The foundation is a local core that knows about servers, sessions, capture, and control. On top of that sit dedicated user interfaces: a desktop TUI for full control and a mobile interface for on-the-go supervision and intervention.

This document combines the previously explored ideas into one review packet: product framing, architecture, interaction model, frontend design, state detection, plugin and permission management, phased delivery, diagrams, and open questions.

---

2. Product Vision

Warren should feel like mission control for a distributed coding workspace.

Instead of remembering which server hosts which agent, SSHing into the right machine, attaching the right tmux session, and reading terminal output one session at a time, the user should be able to open Warren and immediately answer:

- What agents are running right now?
- What is each one working on?
- Which files is each agent touching?
- Which agent is blocked, idle, finished, or asking for help?
- Which plugin or permission setup differs between servers?
- Where do I need to intervene right now?

Warren is not primarily a replacement for tmux or SSH. It is a control layer over them.

It should also not pretend to be a generic cluster manager. The center of gravity is Claude Code-style agent sessions: chat-oriented, tool-using, permission-gated, plugin-extended coding sessions that happen to live inside remote tmux panes.

---

3. Scope and Non-Goals

In scope

- Central registry of remote servers and agent sessions
- Tmux-backed session discovery and management
- Capture of visible and recent session content from tmux panes
- Parsing captured content into chat/activity/file/state summaries
- Detection of actionable states such as waiting for permission, asking a question, or finished
- Central interaction with sessions by sending input back into tmux
- Central management of plugin inventory, enable/disable state, and version drift
- Central management of per-session permission mode and related rules

- Dedicated desktop TUI and mobile-facing frontend surfaces
- A notification system driven by agent-state transitions

Out of scope for the current design

- Full autonomous orchestration of multi-agent workflows
- General-purpose deployment pipeline management
- Replacing existing monitoring stacks like Prometheus or Grafana
- Multi-user collaboration and RBAC-heavy enterprise sharing
- Deep structured extraction from `~/.claude` as a first implementation step

Deferred but expected later

- Reading and indexing session history from `~/.claude`
- Rich file diff review tied to actual pending edits
- Better session replay and timeline views
- More explicit coordination primitives across agents

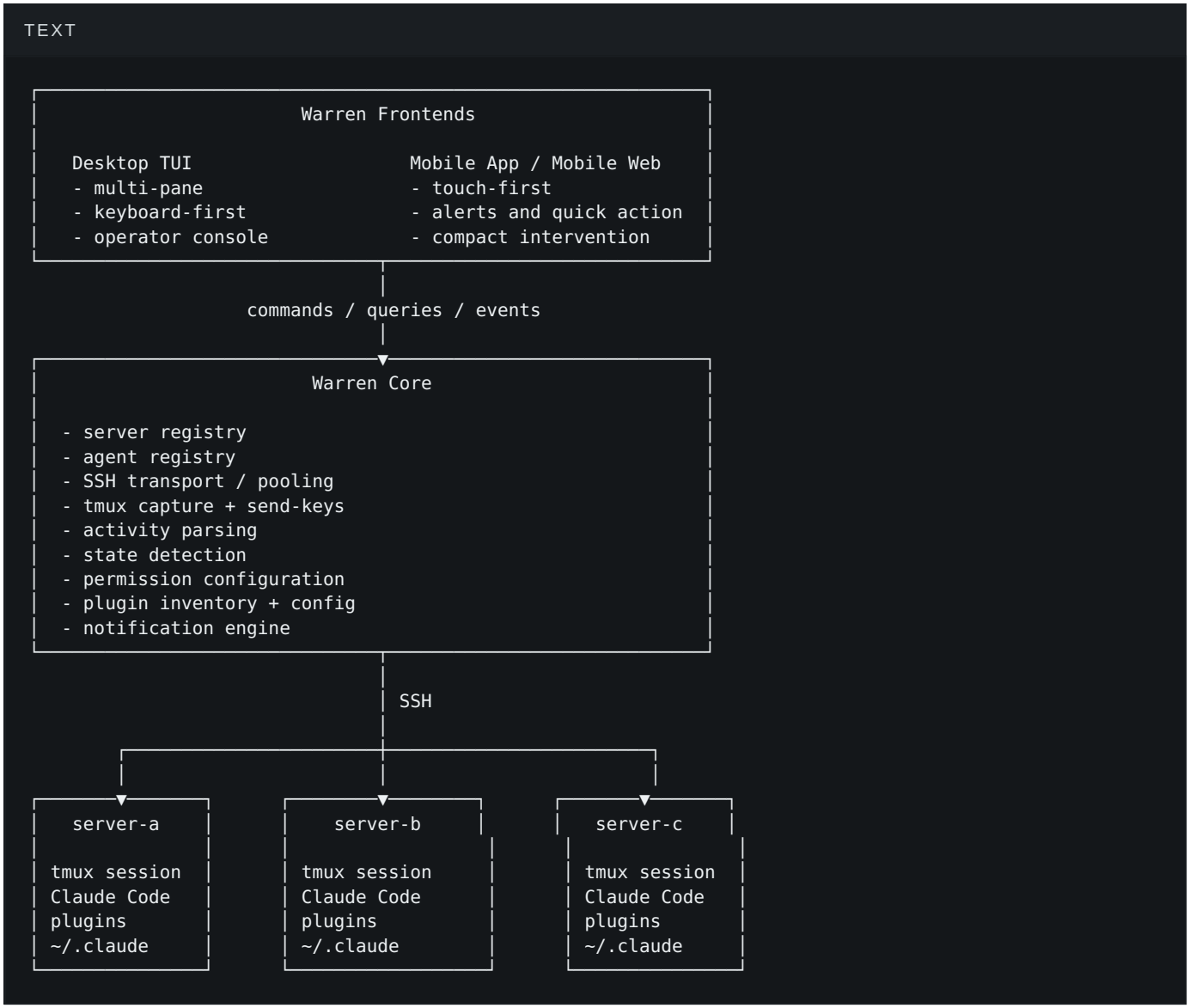
---

4. Core Design Principles

1. **Centralize attention, not execution.** Agents still run remotely, inside tmux, on the machines where they belong. Warren centralizes visibility and control.
2. **Use tmux as the first source of truth.** The first usable version should work by capturing panes and sending keys, because that matches the real runtime environment.
3. **Separate core from interfaces.** Session capture/control logic must not be tangled with terminal or mobile UI code.
4. **Design for intervention.** The most important states are the ones where the user must act: permission prompts, questions, completion, and errors.
5. **Treat plugins and permissions as first-class operational state.** They are part of the workspace topology, not secondary settings.
6. **Make drift visible.** Version mismatches, plugin-state differences, and permission-mode inconsistencies across servers should be easy to spot.
7. **Stay reviewable by humans.** The product must make it easy to inspect what the agent saw, what it did, what file it touched, and why Warren believes the agent is in a given state.

5. System Context

High-level picture



Conceptual model

- A **server** is a remote machine Warren can reach over SSH.
- A **session** is a tmux session that hosts a coding agent.
- An **agent** is Warren's logical model of that session plus its metadata, state, and controls.
- A **capture** is a recent pane snapshot taken from tmux.
- An **activity summary** is a parsed view over one or more captures.
- A **permission profile** is the configuration Warren associates with the session's tool-approval behavior.
- A **plugin profile** is the observed and desired plugin state for a server or session.

6. Core Architecture

6.1 Warren Core

Warren Core is the stable center of the system. It is not a UI. It is a library plus local command surface responsible for remote session supervision.

Its responsibilities are:

- store server definitions,
- store known agent/session mappings,
- connect to remote servers,
- discover or target tmux sessions,
- capture pane content,
- parse captured content,
- detect session state,
- send input back into sessions,
- manage permission settings,
- manage plugin inventory and configuration,
- emit events for notifications and UI updates.

6.2 Remote runtime assumptions

The first design assumes:

- the coding agent already runs inside tmux on a remote server,
- Warren can SSH to that server,
- Warren can run tmux commands there,
- Warren can read relevant Claude Code config/plugin directories,
- Warren may later read `~/.claude` session artifacts directly.

6.3 Interface surfaces

Warren Core supports two frontends:

- **Desktop TUI** as the full operator interface.
- **Mobile interface** as a compact control and intervention surface.

The frontends should consume the same model: agent list, agent details, state, recent activity, permissions, plugins, alerts, and actions.

7. Data and Domain Model

7.1 Core entities

```text
Server
- name
- host
- user
- port
- ssh options
- tags

AgentSession
- logical name
- server reference
- tmux session name
- session type
- created_at
- last_seen_at
- capture policy

AgentActivity
- timestamp
- raw tmux snapshot
- parsed chat excerpts
- parsed file interactions
- parsed tool activity
- pending prompt state
- inferred agent state

PermissionProfile
- mode
- tool rules
- path rules
- inheritance/default source

PluginInventory
- installed plugins per server
- enabled/disabled state
- version
- components exposed
- config overrides

Notification
- type
- source agent
- timestamp
- severity
- action affordances
- read/unread
```

7.2 Storage direction

The current design points toward simple local state for Warren itself, with remote truth fetched from servers.

Likely local storage areas:

- `~/.warren/config.yaml`
- `~/.warren/servers.yaml`

- `~/.warren/registry.json`
- `~/.warren/plugins.json`
- `~/.warren/notifications.json`
- `~/.warren/cache/` for recent captures and parsed activity

The design should preserve a clear distinction between:

- **observed state** from remote servers,
- **desired state** configured in Warren,
- **derived state** such as inferred agent status or attention priority.

---

8. Capture and Parsing Pipeline

8.1 Phase-1 capture source

The first operational source is tmux itself.

Warren captures recent pane content using tmux pane capture on the remote server. The capture window should include:

- currently visible pane content,
- recent scrollback,
- enough lines to reconstruct recent chat and tool activity,
- optionally ANSI escapes if the parser benefits from them.

8.2 Capture pipeline

```
TEXT

remote tmux pane
    ↓
SSH command execution
    ↓
raw pane snapshot
    ↓
normalization
    ↓
parser stages
    ├─ chat extraction
    ├─ file interaction extraction
    ├─ tool activity extraction
    ├─ permission/question detection
    └─ state inference
    ↓
agent activity model
    ↓
frontend views + notifications
```

8.3 Parser outputs

The parser should attempt to extract at least:

- recent user messages,
- recent agent responses,
- files read, written, edited, or referenced,
- commands executed,
- pending permission prompts,
- pending user-choice questions,
- finish/error markers,
- recency of visible activity.

8.4 Parser strategy

The parser will initially be heuristic. It should be explicitly designed as a pipeline with confidence levels rather than one giant regex block.

Recommended parser stages:

1. normalize terminal content,
2. split visible transcript into message/tool/prompt segments,
3. extract structured hints such as paths and tool names,
4. infer current state from recent segments,
5. attach confidence and fall back to raw display when uncertain.

8.5 Future structured source

Later, Warren should support structured session/history ingestion from `~/.claude`.

That future path matters because tmux capture is enough to build the hub, but it is not enough to guarantee perfect reconstruction of full session history, tool metadata, or proposed diffs. The design should therefore keep capture/parsing behind a replaceable interface.

---

9. Agent State Model

9.1 Canonical states

```
TEXT

idle
thinking
executing
waiting_permission
asking_question
finished
error
unknown
```

9.2 State priority

When multiple signals are present, Warren should classify according to user-action priority.

Recommended precedence:

1. `waiting_permission`
2. `asking_question`
3. `error`
4. `finished`
5. `executing`
6. `thinking`
7. `idle`
8. `unknown`

This ensures the hub surfaces the items most likely to need intervention.

9.3 State detection approach

State detection is driven by recent-capture analysis. Signals include:

- visible permission prompts,
- choice/question prompts,
- error strings or obvious failure output,
- explicit completion language,
- tool activity or command execution,
- spinner/progress activity,
- empty prompt plus inactivity.

9.4 State transitions

State changes should be tracked as events, not just overwrites. This enables:

- notification emission,
- activity timelines,
- debugging of parser behavior,
- “what changed” views for operators.

---

10. Interaction Model

The central promise of Warren is not just seeing sessions but acting on them without attaching to tmux directly.

10.1 Passive operations

- see current status,

- inspect recent chat,
- inspect recent file interactions,
- inspect recent commands/tool use,
- inspect plugin state,
- inspect permission mode,
- inspect drift across servers.

10.2 Active operations

- send a message into a session,
- answer a question from a session,
- approve or deny a permission prompt,
- request a file view from the remote workspace,
- open a session directly when full raw interaction is needed,
- change permission mode for a session,
- enable/disable/configure plugins.

10.3 Important safety principle

Warren should not fake certainty when interacting with a live terminal session. Sending input into tmux is powerful but blunt. The design should make it obvious:

- what prompt Warren believes is active,
- what response will be sent,
- whether the response is safe to send,
- whether the captured session changed before the action was applied.

A simple model is: capture → validate prompt still matches → send input → recapture → confirm state transition.

---

11. Permission Management Design

11.1 Requirements

Warren should know, per session:

- current permission mode,
- whether observed behavior matches configured mode,
- tool-level allow/prompt/deny posture,
- path-based exceptions,
- whether a prompt is currently blocking the agent.

11.2 Permission modes

The design assumes Claude Code-style permission modes such as:

- auto,
- default,
- plan,
- acceptEdits,
- dontAsk,
- bypassPermissions.

11.3 Desired vs observed mode

This should be explicit in the UI.

Example:

- desired mode: `default`
- observed mode: `auto`
- drift: yes
- action: reapply / inspect / restart with config

11.4 Permission operations

Warren should support:

- changing mode for one session,
- applying a default profile to multiple sessions,
- managing allow/prompt/deny tool rules,
- managing path-specific rules,
- handling live prompts from the hub.

11.5 Open implementation caution

Changing permission mode at runtime may not be uniformly supported. The design should therefore allow multiple application strategies:

- restart session with new mode,
- mutate remote Claude Code settings,
- invoke an in-session config command if supported.

The product should show which strategy it used.

12. Plugin Management Design

12.1 Requirements

Warren should support:

- plugin discovery on each server,
- per-server inventory,
- per-agent enable/disable status,
- plugin version comparison across servers,
- plugin configuration editing,
- plugin sync and update workflows,
- dependency/conflict inspection.

12.2 Core distinction

There are at least three useful scopes:

- **installed on server**,
- **enabled for session/agent**,
- **configured with overrides at server or agent scope**.

The UI should never collapse these into one ambiguous boolean.

12.3 Operational value

This matters because a session’s capabilities may differ for reasons that are otherwise hard to remember:

- server-a has a newer review plugin,
- server-b disabled a deployment plugin,
- gpu-box has ML-specific plugins unavailable elsewhere,
- an agent is blocked because the needed plugin is disabled.

12.4 Plugin drift review

The plugin surface should be designed partly as a drift-inspection tool:

- which plugins differ,
- which versions are behind,
- which sessions run with non-default config,
- which servers are inconsistent against a chosen baseline.

13. Notification System Design

13.1 Notification triggers

At minimum:

- permission required,
- agent asked a question,
- agent finished,
- agent hit an error,
- agent stopped unexpectedly,
- plugin update/drift detected.

13.2 Priority model

Recommended priority:

- high: permission required, asking question, error,
- medium: finished, stopped,
- low: idle, startup, version drift.

13.3 Delivery model

Desktop TUI:

- in-app attention list,
- badges,
- optionally system notifications.

Mobile:

- in-app inbox,
- push notifications for high/medium priority,
- direct action affordances where safe.

13.4 Notification-centered workflow

A key usage pattern is that a user enters Warren because something needs attention, not because they want to browse. So the notification system should link directly into the relevant session view and action surface.

---

14. Frontend Design Overview

The frontend is not one screen. It is a set of specialized views around four domains:

1. session management,
2. notifications,

3. plugin management,
4. permission management.

The desktop TUI is the primary heavy-duty console. The mobile UI is the compact supervisory/control companion.

---

15. Desktop TUI Design

15.1 Role

The desktop TUI is the main operator console. It should support:

- scanning the whole workspace,
- drilling into one agent,
- resolving prompts quickly,
- comparing plugin/permission state,
- doing bulk administrative actions.

15.2 Primary layout

Recommended shape:

- left pane: agent list / filters,
- main pane: selected agent detail,
- lower or side surfaces: notifications, activity, or detail tabs,
- top-level navigation tabs: Sessions / Plugins / Permissions / Settings.

15.3 Core session view

```
TEXT

┌─ Warren - Sessions ──────────────────────────────────────────┐
│  Agents                          Selected Session            │
│  ┌──────────────────────┐        ┌──────────────────────────┐ │
│  │ backend-api   ⚠      │        │ backend-api @ prod-1     │ │
│  │ ml-trainer    ●      │        │ state: waiting_permission│ │
│  │ frontend      ?      │        │ mode: default            │ │
│  │ worker-3      ✓      │        │                          │ │
│  │ scraper       ✗      │        │ recent chat              │ │
│  └──────────────────────┘        │ files touched            │ │
│                                  │ prompt awaiting answer   │ │
│                                  │ [Approve] [Deny] [Reply] │ │
│                                  └──────────────────────────┘ │
└──────────────────────────────────────────────────────────────┘
```

15.4 Supporting views

- session overview grid,
- agent focus view,
- file viewer,
- plugin browser,

- permission profile editor,
- notification center.

15.5 Interaction style

Keyboard-first with predictable shortcuts:

- navigate lists,
- focus panels,
- approve/deny quickly,
- jump to notifications,
- filter by state/server/tag,
- open direct session attach when needed.

---

16. Mobile Interface Design

16.1 Role

The mobile interface is not a reduced copy of the TUI. It is an interruption-handling tool.

Its primary jobs are:

- tell the user what needs attention,
- let the user inspect the minimum useful context,
- let the user approve/deny/reply quickly,
- let the user check progress while away from a full machine.

16.2 Primary mobile surfaces

- session home list,
- agent detail,
- notification inbox,
- plugins list/detail,
- permissions list/detail.

16.3 Navigation

Single-pane navigation with bottom tabs or a compact menu. The most important entry point is notifications.

16.4 Mobile session card model

Each agent row should include:

- name,
- server,
- state icon,

- last activity age,
- one-line summary,
- one-tap access to the relevant action.

Example:

```
TEXT

● backend-api
prod-1 • 2m ago
Needs permission: edit src/auth.py
[View]
```

16.5 Mobile action model

Mobile should prioritize short, safe actions:

- approve,
- deny,
- reply,
- inspect recent context,
- change mode,
- mute notifications,
- restart or stop only if those controls are explicitly intended.

17. Frontend Information Architecture

17.1 Main areas

```
TEXT

Warren Frontend
├── Sessions
│   ├── All agents
│   ├── Agent detail
│   ├── Chat/activity summary
│   ├── File interactions
│   └── Inline intervention controls
├── Notifications
│   ├── Inbox
│   ├── Per-type settings
│   └── Quick jump to source agent
├── Plugins
│   ├── Server inventory
│   ├── Agent enablement state
│   ├── Version drift
│   └── Config editor
├── Permissions
│   ├── Session mode list
│   ├── Tool rules
│   ├── Path rules
│   └── Bulk apply
└── Settings
    ├── Servers
    ├── Capture behavior
    ├── Notification defaults
    └── UI preferences
```

17.2 Relationship between areas

- **Sessions** are where work is observed.
- **Notifications** are where attention is summoned.
- **Permissions** are where intervention policy is shaped.
- **Plugins** are where capability drift is managed.

These should remain distinct in the UI even if they share backing state.

18. Visualized Core Design

18.1 Core processing diagram

TEXT

18.2 Attention model

TEXT

```
observed session content
        ↓
state detector
        ↓
Does user need to act?
        ├─ no  → update status only
        └─ yes → create notification + expose action controls
```

18.3 Control loop

```
TEXT

capture current prompt
     ↓
parse and identify state
     ↓
user chooses action
     ↓
validate prompt still current
     ↓
send keys into tmux session
     ↓
recapture session
     ↓
confirm state transition / report ambiguity
```

---

19. Visualized Frontend Designs

19.1 Desktop TUI overview

```
TEXT

┌──────────────────────────────── Warren ────────────────────────────────┐
│ [Sessions] [Plugins] [Permissions] [Notifications] [Settings]          │
├──────────────────────┬─────────────────────────────────────────────────┤
│ Agent List           │ Detail Pane                                     │
│                      │                                                 │
│ ⚠ backend-api        │ backend-api @ prod-1                            │
│ ● ml-trainer         │ mode: default                                   │
│ ? frontend           │ state: waiting_permission                       │
│ ✓ worker-3           │                                                 │
│ ✗ scraper            │ recent chat                                     │
│                      │ files touched                                   │
│ Filters              │ pending prompt                                  │
│ - by server          │ [Approve] [Deny] [Reply] [Open File]            │
│ - by state           │                                                 │
│ - by tag             │                                                 │
├──────────────────────┴─────────────────────────────────────────────────┤
│ Notification strip: backend-api needs permission • frontend asked Q    │
└────────────────────────────────────────────────────────────────────────┘
```

19.2 Desktop plugin management

```
TEXT

┌──────────────────────────── Plugins ────────────────────────────────┐
│ Server tree                Selected plugin                          │
│                            code-review                              │
│ prod-1                     version drift: prod-2 behind             │
│   ✓ code-review            enabled on: backend-api, frontend        │
│   ✓ deployment             disabled on: scraper                     │
│   ✗ testing-utils          actions: [Enable] [Disable] [Update]     │
│                            config scopes: global/server/agent       │
└─────────────────────────────────────────────────────────────────────┘
```

19.3 Desktop permission management

```
TEXT

┌──────────────────────────── Permissions ──────────────────────────────┐
│ Agent list                  Selected profile                          │
│ backend-api  default        mode: default                             │
│ ml-trainer   auto           tool rules                                │
│ frontend     plan           - Read: allow                             │
│                             - Edit: prompt                            │
│                             - Bash: deny                              │
│                             path rules                                │
│                             - src/**/*.py allow edit                  │
│                             - .env* deny all                          │
│                             [Save] [Apply to many]                    │
└───────────────────────────────────────────────────────────────────────┘
```

19.4 Mobile overview

```
TEXT

┌──────────────────────────────┐
│ Warren              🔔 3     │
├──────────────────────────────┤
│ ⚠ backend-api                │
│ prod-1 • needs permission    │
│ [View] [Approve]             │
│                              │
│ ? frontend                   │
│ prod-2 • asked question      │
│ [View] [Reply]               │
│                              │
│ ● ml-trainer                 │
│ gpu-box • executing          │
│ [View]                       │
├──────────────────────────────┤
│ Sessions  Plugins  Perms     │
└──────────────────────────────┘
```

19.5 Mobile agent detail

```
TEXT

┌──────────────────────────────┐
│ ← backend-api                │
├──────────────────────────────┤
│ state: waiting_permission    │
│ mode: default                │
│                              │
│ recent chat                  │
│ - User: Fix auth bug         │
│ - Agent: I found the issue   │
│                              │
│ file: src/auth.py            │
│ action: pending edit         │
│                              │
│ [View File]                  │
│ [Approve] [Deny]             │
│ [Reply]                      │
└──────────────────────────────┘
```

20. Operational Flows

20.1 Permission-response flow

1. Warren captures session output.
2. Parser detects a permission prompt.
3. Agent state becomes `waiting_permission`.
4. Notification is emitted.
5. User opens agent detail from notification.
6. Warren shows recent chat and referenced file context.
7. User approves or denies.
8. Warren validates prompt freshness and sends response.
9. Warren recaptures and verifies the session progressed.

20.2 Question-response flow

1. Agent asks a design or implementation question.
2. Parser marks state as `asking_question`.
3. Notification is emitted.
4. User opens context.
5. User chooses a quick option or sends a reply.
6. Warren injects the response into tmux.
7. Warren confirms the session resumed work.

20.3 Plugin-drift review flow

1. Warren scans plugins across servers.
2. Version/state differences are recorded.
3. Plugin dashboard highlights mismatches.
4. User inspects one plugin.
5. User updates, syncs, or changes enablement scope.
6. Warren applies the action and records new observed state.

20.4 Permission-profile maintenance flow

1. User inspects a session repeatedly asking for the same class of action.
2. User opens permission management.
3. User edits the session's permission profile or applies a broader profile.
4. Warren applies via restart, remote settings mutation, or in-session config.
5. Warren marks whether observed mode now matches desired mode.

21. Phased Delivery

Phase 1 — Central Read-Only Hub

Goal: make the distributed workspace visible from one place.

Includes:

- server/session registry,
- tmux capture,
- activity parsing,
- state detection,
- session overview UI,
- agent detail UI,
- basic notification center,
- file-interaction summaries.

This phase succeeds if the user can open Warren and understand what each agent is doing and which ones need attention.

Phase 2 — Interactive Session Control

Goal: act on sessions from the hub.

Includes:

- send messages into sessions,
- answer questions,
- approve/deny permission prompts,
- open remote file viewer,
- recapture/validate control loop,
- tighter notification-to-action flow.

This phase succeeds if the user rarely needs to attach directly to tmux for routine intervention.

Phase 3 — Permission and Plugin Control Planes

Goal: make operational configuration manageable centrally.

Includes:

- per-session permission profiles,
- mode drift visualization,
- plugin inventory and version drift,
- enable/disable/configure plugin actions,
- bulk management flows.

This phase succeeds if the workspace's behavioral differences across servers are visible and controllable.

Phase 4 — Structured Claude Data Integration

Goal: replace or augment heuristic capture with structured session history.

Includes:

- reading `~/.claude` artifacts,
- richer conversation history,
- more accurate file/action views,
- better diff and replay support,
- more reliable prompt/state classification.

This phase succeeds if Warren becomes both more accurate and more reviewable than tmux-only capture.

---

22. Risks and Design Tensions

22.1 Tmux parsing fragility

The biggest risk is that terminal capture is heuristic. Output formatting can shift, themes may vary, and partial content may confuse the parser.

Mitigation:

- confidence scores,
- raw-output fallback,
- state history for debugging,
- replaceable parser architecture,
- later `~/.claude` integration.

22.2 Input injection ambiguity

Sending a response into a live terminal session is powerful but dangerous if the session state changed between capture and action.

Mitigation:

- prompt validation before send,
- quick recapture after send,
- visible "prompt no longer matches" failures,
- conservative default for destructive actions.

22.3 Permission-mode truth

Observed behavior and configured intent may diverge. A session might not immediately reflect Warren's desired settings.

Mitigation:

- model desired vs observed separately,
- surface drift explicitly,
- record application strategy.

22.4 Plugin-state ambiguity

Installed, enabled, and configured are different things. If Warren blurs them, the UI becomes misleading.

Mitigation:

- distinct fields and views for inventory, enablement, and config,
- scope-aware UI labels.

22.5 Mobile surface creep

A mobile app can easily become an awkward clone of the desktop tool.

Mitigation:

- keep mobile optimized for alerts, inspection, and short interventions,
- reserve deep administrative work for the desktop TUI.

---

23. Open Questions

These are the design questions that still matter most.

1. **How reliable can phase-1 parsing be in real Claude Code sessions?**
2. **What is the exact safest control loop for sending input into tmux?**
3. **How should Warren distinguish current prompt context from stale visible text?**
4. **What is the best source of truth for permission mode: tmux-observed behavior, remote settings, or both?**
5. **How should Warren apply permission changes to running sessions when runtime config is not supported?**
6. **How much plugin state is session-scoped versus server-scoped in practice?**
7. **Should the mobile interface be a PWA first or a native shell over a web app?**
8. **When Warren later reads `~/.claude`, which data path becomes authoritative and which remains fallback?**
9. **What minimum capture frequency gives a responsive hub without creating too much SSH overhead?**
10. **How should Warren display confidence/uncertainty when state inference is ambiguous?**

---

24. Recommended Design Direction

The strongest path forward is:

- keep the product centered on the **central hub** idea,

- treat **tmux capture + send-keys** as the first practical integration layer,
- build **desktop TUI first** as the main operator console,
- keep **mobile** focused on attention and intervention,
- make **notifications, permissions, and plugins** first-class surfaces rather than secondary settings,
- preserve a clean seam for future structured ingestion from `~/.claude`.

In short: Warren should first become the place where the user understands and supervises the whole distributed workspace. Only after that foundation is solid should it become smarter, deeper, and more automated.

---

25. Review Checklist

This is the checklist a human reviewer should use while reading this design.

- Does the central-hub framing match the real product intent?
- Is the separation between core, TUI, and mobile clear enough?
- Are the phase boundaries sensible?
- Does the state model capture the real attention states?
- Are permission and plugin management treated at the right level of importance?
- Does the interaction model feel safe enough for live session control?
- Are the frontend surfaces appropriately distinct rather than over-combined?
- Are the open questions the real blockers, or are any missing?

---

26. Appendix: Short Positioning Statement

Warren is a central supervisory interface for distributed Claude Code sessions running inside remote tmux environments. It turns a fragmented SSH-and-tmux workspace into one visible, actionable control plane, starting with capture and intervention via tmux and evolving toward deeper session awareness through Claude-local metadata.

Source: `AGGREGATE-DESIGN-REVIEW.md`